

## LAB # 13

### Construct java programs using the concepts polymorphism and abstract classes

#### Objective

To study Method Overriding(polymorphism) and Abstract class in java.

#### Lab Goals

- Introduction to Polymorphism
- How to construct method overriding?
- Introduction to abstraction
- How to construct abstract class in Java.

#### **Introduction to Polymorphism**

Polymorphism describes a language's ability to process objects of various types and classes through a single, uniform interface. Polymorphism in Java has two types: Compile time polymorphism (static binding) and Runtime polymorphism (dynamic binding). Method overloading is an example of static polymorphism, while method overriding is an example of dynamic polymorphism. Polymorphism is the ability of an object to take on many forms. The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object.

**Runtime Polymorphism (Method Overriding):** In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to override the method in the superclass. When an overridden method is called from within a subclass, it will always refer to the version of that method defined by the subclass. The version of the method defined by the superclass will be hidden. Consider the following:

#### **Syntax demonstrated using an Example Code:**

```
// Superclass
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

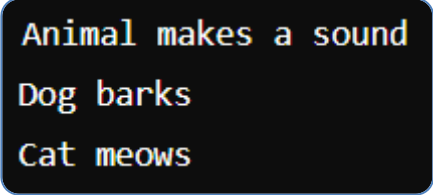
// Subclass Dog
class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks");
    }
}
```

```
// Subclass Cat
class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("Cat meows");
    }
}

// Main class to test polymorphism
public class PolymorphismExample {
    public static void main(String[] args) {
        // Creating instances of Animal, Dog, and Cat
        Animal myAnimal = new Animal(); // Animal reference and object
        Animal myDog = new Dog();       // Animal reference but Dog object
        Animal myCat = new Cat();       // Animal reference but Cat object

        // Calling the makeSound method on each object
        myAnimal.makeSound(); // Output: Animal makes a sound
        myDog.makeSound();    // Output: Dog barks
        myCat.makeSound();    // Output: Cat meows
    }
}
```

**Output:**



```
Animal makes a sound
Dog barks
Cat meows
```

## Introduction of Abstraction

Abstraction in Java is a concept of hiding the complex implementation details and showing only the essential features of an object. It allows us to reduce programming complexity and effort. By using abstraction, a developer can focus on what an object does instead of how it does it. In Java, abstraction is achieved through **abstract classes** and **interfaces**.

## Abstract Class in Java

An abstract class is a class that cannot be instantiated on its own and must be subclassed. It can contain abstract methods (methods without a body) as well as concrete methods (methods with a body). The abstract methods must be implemented by the subclasses.

**Key Points:**

1. An abstract class is declared using the abstract keyword.
2. It can have both abstract and non-abstract methods.
3. Abstract methods are declared without an implementation.
4. An abstract class cannot be instantiated directly.

**Example Code:**

```
// Abstract class
abstract class Animal {
    // Abstract method (does not have a body)
    public abstract void makeSound();

    // Regular method
    public void sleep() {
        System.out.println("The animal is sleeping");
    }
}

// Subclass (inherits from Animal)
class Dog extends Animal {
    public void makeSound() {
        System.out.println("The dog says: bow wow");
    }
}

class Cat extends Animal {
    public void makeSound() {
        System.out.println("The cat says: meow");
    }
}

public class Main {
    public static void main(String[] args) {
        // Create Dog and Cat objects
        Dog myDog = new Dog();
        Cat myCat = new Cat();

        // Call the abstract method
        myDog.makeSound();
        myCat.makeSound();

        // Call the regular method
        myDog.sleep();
        myCat.sleep();
    }
}
```

**Output:**

```
The dog says: bow wow  
The cat says: meow  
The animal is sleeping  
The animal is sleeping
```

**Exercise**

- A. Create a class called computers and two classes MyComputer and YourComputer which inherits computer class. Define appropriate fields for the three classes. Create another class processor as a composite class of computer. Write a method which prints the differences between the processors of two computers.
- B. Create a hierarchy of Liquid types. The parent class is Liquid, and its children classes are Coffee and Milk. A Liquid object will have an Add() method that takes some amount of liquid from user and adds it to the original amount of liquid, a remove () method that remove some amount of liquid, and a removeAll() method that removes all amount of liquid. Each subtype of Liquid overrides the three methods of liquid class. Coffee class have some specialize behavior that is it has a Swirl() method that prints “Swirling coffee” Create a test class that create an array of Liquid class having ten elements and populate it randomly with objects (all classes) and Call there method to show polymorphic behavior. Note: Each of the methods in the hierarchy classes has to be called in the Test class. Using for loop and use the InstanceOf() method to show Coffee class specialized Behavior.
- C. Create a class purse, which contains hidden data, rupees and paisas. The class contains 2 constructors, one with zero argument that set the data to Zero, and others set the data provided by the arguments. Instance method that reset data to Zero should be provided. Method for conversion of rupees into paisas should be provided. Also it must contain addmoney, deletemoney methods.
- D. You are given an abstract class Shape that defines a blueprint for shapes with an abstract method calculateArea(). Your task is to:
  - Create two subclasses, Circle and Rectangle, that extend the Shape class.
  - Implement the calculateArea() method in both subclasses.
  - Create instances of Circle and Rectangle in a Main class and display their areas.